



Rutex GO

- TURISMO GAMIFICADO -

- MANUAL TECNICO -

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026	Tutor/a: María Mercedes Martínez Frago	

Índice

- 1. Descripción de lo implementado (PMV)..... 3
 - 1.1. Propósito del Proyecto 3
 - 1.2. Alcance de la Evaluación Actual (PMV) 3
 - 1.3. Funcionalidades 100% Operativas..... 3
- 2. Arquitectura y Módulos del Sistema 4
 - 2.1. Modelo de Arquitectura: Clean Architecture 4
 - 2.2. Diagrama Estructurado del Sistema (Flujo Lineal)..... 4
 - 2.3. Responsabilidad de los Módulos Implementados..... 5
- 3. Dependencias y Versiones 6
 - 3.1. Entorno de Desarrollo Base 6
 - 3.2. Librerías del Proyecto 6
 - 3.3. Requisitos del Sistema 7
- 4. Estructura del Repositorio 8
 - 4.1. Árbol de Directorios Detallado 8
 - 4.2. Descripción de Componentes Clave 9
- 5. Funcionalidades Principales y Lógica de Datos 9
 - 5.1. Autenticación y Gestión de Sesión (Módulo Auth) 9
 - 5.2. Motor de Exploración (Módulo Routes) 10
 - 5.3. Sistema de Validación y Gamificación (Módulo Mission)..... 10
- 6. Base de Datos (Modelo de Datos NoSQL) 11
 - 6.1. Colección: usuarios..... 11
 - 6.2. Colección: config_rangos..... 12
 - 6.3. Colección: ciudades 13
 - 6.4. Colección: rutas 13
 - 6.5. Colección: puntos_interes 14
 - 6.6. Colección: misiones 15
 - 6.7. Colección: resultado 15

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

6.8. Modelo Entidad-Relación (ERD) 16

6.9. Migraciones y Gestión de Esquema 16

6.10. Seed y Datos de Prueba..... 17

7. Pruebas del sistema 17

7.1 Objetivos de las pruebas 17

7.2 Entorno de pruebas 18

7.3 Tipos de pruebas realizadas 18

7.4 Evidencias de las pruebas..... 21

8. Problemas conocidos y limitaciones actuales 21

8.1 Integración del mapa..... 21

8.2 Ajustes de cálculos lógicos del sistema 22

8.3. Persistencia y sincronización de datos 22

8.4 Validación de QR en entorno real..... 23

8.5 Sistema de progresión y rangos..... 23

8.6 Limitaciones del uso de Firebase..... 24

9.Próximos pasos para la defensa final..... 24

9.1 Integración del sistema de navegación con mapas 24

9.2 Optimización del sistema de puntuación y progreso 25

9.3 Implementación completa del sistema de rangos..... 25

9.4 Mejoras en la persistencia de datos 25

9.5 Mejoras en la experiencia de usuario..... 26

9.6 Preparación de la demostración final..... 26

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García		13/03	V 1
	0492 · Proyecto DAM · 2025/2026		Tutor/a: María Mercedes Martínez Fragoso	

1. Descripción de lo implementado (PMV)

1.1. Propósito del Proyecto

RutexGo nace como una solución digital ante la rigidez del turismo tradicional en ciudades con alto patrimonio cultural, como Mérida. El proyecto busca captar al público joven y familias que no encajan en las visitas guiadas convencionales, ofreciendo una alternativa gamificada, educativa y sostenible.

El nombre sintetiza la visión del producto: **Ruta** (itinerario interactivo), **eX** (referencia a Extremadura como eje inicial) y **Go** (acción, movimiento y dinamismo digital).

1.2. Alcance de la Evaluación Actual (PMV)

Para esta primera entrega, nos hemos centrado en crear un prototipo sólido y funcional en **Mérida**, que es nuestra base de operaciones. No es solo un diseño visual; hemos conectado la aplicación con una infraestructura real en la nube para que todo lo que haga el usuario se guarde y tenga sentido.

El objetivo principal de este PMV es que el usuario pueda registrarse, elegir una ruta y empezar a "jugar" con la ciudad de forma autónoma.

1.3. Funcionalidades 100% Operativas

A continuación, se detallan los módulos que se encuentran plenamente integrados con la base de datos y funcionales para el usuario final:

- **Sistema de Gestión de Usuarios (Firebase Auth):** Implementación completa de los flujos de registro, inicio de sesión y persistencia de sesión. El sistema genera un "Home" personalizado que reconoce el perfil del usuario activo.
- **Catálogo Dinámico de Ciudades y Rutas:** Módulo de consulta que extrae en tiempo real desde la base de datos las rutas disponibles, segmentadas por ciudad, con sus respectivos monumentos y tiempos estimados.
- **Validación de Presencia por QR:** Integración del hardware de la cámara para la lectura de códigos QR. Esta función actúa como disparador de eventos, verificando que el usuario se encuentra físicamente en el monumento antes de permitir el progreso.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

- **Módulo de Gamificación (Quiz):** Sistema de lógica de preguntas y respuestas asociado a cada hito de la ruta. Los retos se activan tras la validación por QR, permitiendo una experiencia de aprendizaje interactiva.
- **Motor de Progresión y Rangos:** Lógica de negocio que gestiona la jerarquía de niveles del usuario. El sistema procesa los aciertos del Quiz para actualizar el estatus histórico del jugador, permitiendo evolucionar desde el rango de "Esclavo" hasta el de "Emperador".
- **Interfaz Orientada a la Sostenibilidad:** Diseño de experiencia de usuario (UX) enfocado en la eliminación de mapas físicos de papel y la promoción de rutas peatonales saludables, reduciendo la huella de carbono del turista.

2. Arquitectura y Módulos del Sistema

2.1. Modelo de Arquitectura: Clean Architecture

Para el desarrollo de **RuteX Go**, hemos implementado el patrón de **Clean Architecture** (Arquitectura Limpia). Esta decisión nos permite separar la lógica de negocio de las herramientas externas (como Firebase o la cámara), garantizando que el código sea escalable, fácil de testear y robusto.

La arquitectura se organiza en un flujo lineal de dependencias, donde la interfaz de usuario nunca accede directamente a la base de datos, sino que lo hace a través de capas intermedias de control.

2.2. Diagrama Estructurado del Sistema (Flujo Lineal)

El flujo de información en la aplicación sigue este orden jerárquico:

1. CAPA DE PRESENTACIÓN (Flutter UI + Provider)

- **Vistas:** Pantallas de catálogo de rutas, visor de mapas y módulos de juego.
- **Gestor de Estado (Provider):** Notifica a la interfaz los cambios en tiempo real (ej: actualización del rango de "Esclavo" a "Emperador").
- *Relación:* Envía las acciones del usuario hacia la lógica de negocio.

2. CAPA DE DOMINIO (Casos de Uso y Entidades)

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

- **Casos de Uso:** Contiene las reglas del negocio (ej: LoginUseCase, ValidateQRUseCase, AddPointsUseCase).
- **Entidades:** Define los modelos de datos puros (qué atributos tiene una Ruta o un Usuario).
- **Relación:** Es el "cerebro" que decide qué debe pasar sin saber cómo funcionan los servidores.

3. CAPA DE DATOS (Repositorios y Data Sources)

- **Repositorios:** Actúan como intermediarios que traducen las peticiones del dominio al lenguaje de Firebase.
- **Relación:** Recupera la información física y digital del entorno.

4. INFRAESTRUCTURA EXTERNA (Firebase Backend)

- **Firebase Auth:** Servicio de seguridad para la gestión de identidad.
- **Cloud Firestore:** Base de datos NoSQL donde se almacena las distintas colecciones de datos como por ejemplo: ciudades, usuarios, rutas, monumentos, etc

2.3. Responsabilidad de los Módulos Implementados

Para que el desarrollo fuera organizado, dividimos **RuteX Go** en bloques donde cada uno tiene una "misión" muy clara. Así es como se reparten las tareas:

- **Módulo Auth (Identidad):** Su único trabajo es saber quién eres. Se encarga de que puedas registrarte, entrar con tu cuenta y que tus puntos no se pierdan al cerrar la app. Es el guardián de la seguridad usando Firebase.
- **Módulo Routes (El Selector):** Es el catálogo inteligente. Su responsabilidad es filtrar el contenido: tú eliges una ciudad y este módulo "limpia" la base de datos para traerte solo las rutas que pertenecen a ese lugar (como Mérida). Es el paso previo antes de empezar la aventura.
- **Módulo Mission (La Experiencia Viva):** Este es el motor de la app y el que más tecnología agrupa. Tiene tres tareas críticas que funcionan en cadena:
 1. **Mapa de navegación:** Te guía físicamente por la ciudad usando GPS para que sepas dónde está el siguiente monumento.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

2. **Validación por QR:** Controla la cámara para escanear el código físico del monumento. Es el que dice: "Vale, estás aquí de verdad, puedes seguir".
 3. **Sistema Quiz:** Una vez validado el QR, lanza las preguntas interactivas. Gestiona los aciertos y errores para decidir si has superado la prueba.
- **Módulo Profile (Progresión):** Es el contador de tu viaje. Recibe los resultados del módulo de misiones y actualiza tu estatus. Se encarga de la lógica de rangos: sabe cuántos puntos te faltan para dejar de ser un "Esclavo" y convertirte en "Emperador".
 - **Módulo Core (La Base):** Es la infraestructura que no se ve pero que mantiene todo en pie. Aquí definimos los colores, el diseño sostenible, cómo se navega de una pantalla a otra y cómo avisar al usuario si se queda sin internet o hay un error.

3. Dependencias y Versiones

El desarrollo de **RuteX Go** se apoya en un ecosistema de herramientas modernas y de código abierto que garantizan la estabilidad y la rapidez de la aplicación.

3.1. Entorno de Desarrollo Base

- **Lenguaje:** Dart (optimizado para interfaces rápidas).
- **Framework:** Flutter (Canal estable).
- **Backend (BaaS):** Firebase (Google Cloud).

3.2. Librerías del Proyecto

Estas son las dependencias clave que hemos integrado para que las funcionalidades del PMV sean posibles:

Dependencia	Versión	Función en RutexGo
firebase_core / auth	^4.4.0 / ^6.1.4	Gestión de usuarios y seguridad del login.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

Dependencia	Versión	Función en RutexGo
cloud_firestore	^6.1.2	Nuestra base de datos en tiempo real para rutas y puntos.
mobile_scanner	^7.2.0	Control de la cámara para la lectura de los códigos QR.
flutter_map	^7.0.2	El motor que renderiza el mapa interactivo de la ciudad.
geolocator	^14.0.2	Obtiene la ubicación del turista para guiarlo en la ruta.
provider	^6.1.5	Gestiona el estado de la app (puntos, niveles, sesión).
flutter_joystick	^0.2.2	Control interactivo para mejorar la UX en los minijuegos.
intl	^0.20.2	Formateo de textos, fechas y números.
http	^1.6.0	Para futuras conexiones con servicios externos o APIs de terceros.

3.3. Requisitos del Sistema

Para que la aplicación funcione correctamente durante las pruebas de esta evaluación, se requiere:

- **Android:** API 21 (Lollipop) o superior.
- **iOS:** Versión 12.0 o superior.
- **Permisos necesarios:** La app solicitará acceso a la **Cámara** (para el QR) y a la **Ubicación** (para el mapa).

4. Estructura del Repositorio

Para el desarrollo de **RuteX Go**, hemos organizado el proyecto de manera que la documentación, los activos de marca y el código fuente estén claramente separados. Esto facilita tanto el despliegue técnico como la gestión académica del proyecto.

4.1. Árbol de Directorios Detallado

A continuación, se muestra la jerarquía de carpetas del repositorio con su respectiva función dentro del ecosistema de la app:

```

rutex-go-project/
├── .gitignore           # Archivos omitidos por Git
├── LICENSE             # Licencia del software
├── README.md           # Descripción general
├── tablas proyecto final.jpg # Esquema de base de datos
├── assets/             # Recursos estáticos
│   └── rangos/         # Iconografía (Esclavo a Emperador)
├── docs/               # Documentación del ciclo de vida
│   ├── design/         # Prototipos UI/UX
│   ├── images/         # Capturas y diagramas
│   ├── pdf/            # Entregables oficiales
│   ├── project-proposal.md # Propuesta de viabilidad
│   └── technical_documentation.md # Documentación técnica
├── mobile_app/         # Proyecto Flutter
│   ├── android/        # Configuración nativa Android
│   ├── ios/            # Configuración nativa iOS
│   ├── assets/         # Recursos internos de la app
│   ├── firebase.json   # Vinculación con Firebase
│   ├── pubspec.yaml    # Dependencias y versiones
│   └── lib/            # Núcleo del código fuente
│       ├── core/       # Elementos transversales
│       │   ├── constants/
│       │   ├── error/
│       │   ├── map/
│       │   ├── routes/
│       │   ├── theme/
│       │   ├── utils/
│       │   └── widgets/
│       └── features/    # Módulos funcionales
│           ├── auth/    # Login y Registro
│           ├── mission/ # QR, Mapa y Quiz
│           ├── profile/ # Rangos y Puntos
│           ├── routes/  # Selección de rutas
│           └── splash/  # Pantalla de carga

```

Ilustración 1: Estructura Repositorio

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

4.2. Descripción de Componentes Clave

- **assets/rangos/**: Es una de las carpetas más importantes para la gamificación, ya que contiene los logos que visualiza el usuario conforme evoluciona en su rango histórico.
- **docs/**: A diferencia de otros proyectos, hemos mantenido un histórico de la evolución del diseño y las propuestas en formato Markdown y PDF, asegurando que la trazabilidad del proyecto sea transparente.
- **mobile_app/lib/**: Siguiendo el patrón **Clean Architecture**, esta carpeta es el núcleo técnico donde se separan las responsabilidades en capas de Datos, Dominio y Presentación para cada funcionalidad (features).
- **tablas proyecto final.jpg**: Archivo de referencia rápida que contiene el esquema de la base de datos Firestore, permitiendo a cualquier desarrollador entender la estructura NoSQL sin entrar en la consola de Google.

5. Funcionalidades Principales y Lógica de Datos

Al utilizar una infraestructura basada en **Firebase (BaaS)**, la aplicación no consume una API REST convencional, sino que interactúa directamente con los servicios de Google mediante el SDK oficial. A continuación, se detallan las funciones críticas del sistema y la lógica de datos que siguen.

5.1. Autenticación y Gestión de Sesión (Módulo Auth)

La seguridad se gestiona mediante **Firebase Authentication**, permitiendo un flujo de acceso seguro y persistente.

- **Registro/Login**: La app envía las credenciales mediante el caso de uso AuthRepository. Firebase devuelve un UserToken que la app almacena localmente para mantener la sesión activa.
- **Persistencia**: Al iniciar la app, el módulo Splash verifica si existe un usuario activo antes de redirigir a la pantalla principal.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

5.2. Motor de Exploración (Módulo Routes)

Este componente gestiona la visualización del catálogo turístico basándose en la ubicación geográfica seleccionada.

- **Filtrado por Ciudad:** Se realiza una consulta (Query) a la colección /cities para listar las opciones. Una vez seleccionada, se ejecuta una petición a /routes filtrando por el ID de la ciudad elegida.
- **Estructura de Datos:** Cada ruta devuelve un listado de monumentos que incluye:
 - Coordenadas (Latitud/Longitud).
 - Información histórica y multimedia.
 - ID de validación para el escáner.

5.3. Sistema de Validación y Gamificación (Módulo Mission)

Es la funcionalidad central del PMV y la que requiere interacción directa con el hardware del dispositivo.

- **Validación de Presencia (QR):** Cuando el usuario escanea un código físico, el módulo mobile_scanner extrae el valor del QR. El caso de uso executeScan (getPuntoByQr) compara este valor con el campo qr_code_id almacenado en Firestore para ese monumento específico.
- **Lógica del Quiz:** Si el QR es válido, se descargan las preguntas asociadas al monumento. La lógica de aciertos/errores se procesa en el cliente y, al finalizar, se dispara una actualización en el perfil del usuario:
 - **Actualización de Puntos:** Se incrementa el campo total_points en el documento del usuario.
 - **Cálculo de Rango:** Se dispara una función que verifica si los nuevos puntos alcanzan el umbral del siguiente nivel (ej.: de *Esclavo* a *Liberto*).

6. Base de Datos (Modelo de Datos NoSQL)

Para **RuteX Go** hemos diseñado una estructura en **Cloud Firestore** que prioriza la velocidad de lectura y la facilidad para escalar. Al usar colecciones raíz desacopladas (sin relaciones rígidas), la aplicación puede crecer en usuarios y contenido sin pérdida de rendimiento.

Ventajas clave:

- **Independencia:** Cada módulo (Rutas, Usuarios, Misiones) funciona de forma autónoma, facilitando el mantenimiento.
- **Flexibilidad:** Permite añadir nuevas ciudades o mecánicas de juego sin reestructurar la base de datos.
- **Eficiencia:** Optimizada para el consumo móvil con sincronización en tiempo real y soporte offline.

6.1. Colección: usuarios

Es la colección principal encargada de almacenar el perfil, el progreso y los permisos de cada turista en la plataforma.

Campo	Tipo de Dato	Descripción
email	String	Correo electrónico vinculado a la cuenta de Firebase Auth.
nombre	String	Nombre o alias del usuario para mostrar en la interfaz.
puntos	Number	Acumulado total obtenido mediante la resolución de Quizzes.
rango	String (Ref.)	ID del documento en la colección config_rangos que define el nivel actual.
rutas_completadas	Array (Strings)	Lista de IDs de los documentos en la colección rutas que el usuario ya ha finalizado.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

Campo	Tipo de Dato	Descripción
isAdmin	Boolean	Flag de seguridad para distinguir entre usuarios turistas y administradores.
fecha_creacion	Timestamp	Registro de cuándo se registró el usuario en RuteX Go.
ultimo_acceso	Timestamp	Control de la última vez que el usuario inició sesión.

6.2. Colección: config_rangos

Esta colección funciona como el motor de niveles del juego. Define los umbrales de puntuación y la identidad visual de cada rango histórico disponible. Al ser un Array de Mapas, permite que la app recorra todos los rangos fácilmente para calcular cuál le toca al usuario según sus puntos.

Campo (dentro del Array)	Tipo de Dato	Descripción
logo	String (URL)	Enlace al recurso gráfico (alojado en Firebase Storage) que se muestra en el perfil.
nombre	String	Título del rango (ej: Esclavo, Libertado, Senador, etc.).
puntos_necesarios	Number	Cantidad mínima de puntos que el usuario debe acumular para desbloquear este nivel.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

6.3. Colección: ciudades

Actúa como el catálogo geográfico de la aplicación. Permite segmentar las rutas y puntos de interés según la localidad donde se encuentre el turista.

Campo	Tipo de Dato	Descripción
nombre	String	Nombre de la localidad (ej.: Mérida, Badajoz).
provincia	String	Provincia a la que pertenece para facilitar búsquedas regionales.
imagen	String (URL)	Fotografía de portada que se muestra en el selector de ciudades de la app.
isActive	Boolean	Control de visibilidad. Indica si la ciudad tiene contenido publicado y disponible para los usuarios.

6.4. Colección: rutas

Define los itinerarios temáticos disponibles en la aplicación. Esta colección actúa como nexo de unión entre una ciudad y los monumentos o paradas que el usuario debe visitar.

Campo	Tipo de Dato	Descripción
nombre	String	Título descriptivo de la ruta (ej: Ruta Monumental Romana).
id_ciudad	String (Ref)	Identificador del documento de la colección ciudades al que pertenece.
id_puntos_interes	Array (Strings)	Lista ordenada de IDs que referencian a los documentos de la colección puntos_interes.
dificultad	String	Nivel de esfuerzo estimado para el turista (Fácil, Media, Alta).
duracion	String	Tiempo estimado para completar el recorrido.

Campo	Tipo de Dato	Descripción
puntos_totales	Number	Suma de los puntos que el usuario obtendrá al completar todas las misiones de la ruta.
isActive	Boolean	Estado de publicación de la ruta en la interfaz de usuario.

6.5. Colección: puntos_interes

Contiene la información detallada de cada monumento o sitio de interés histórico. Es la base que utiliza el mapa interactivo y el sistema de validación por cámara.

Campo	Tipo de Dato	Descripción
nombre	String	Nombre oficial del monumento (ej: Anfiteatro Romano).
descripcion	String	Texto histórico y educativo que el usuario lee al llegar al punto.
localización	Geopoint	Coordenadas exactas (Latitud y Longitud) para posicionar el marcador en el mapa.
imagen	String (URL)	Fotografía en alta resolución del monumento.
qr_code	String	Código identificador único que el sistema busca al escanear el QR físico.
radio_activacion	Number	Distancia en metros (ej: 50m) dentro de la cual el usuario puede desbloquear la misión.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Frago		

6.6. Colección: misiones

Esta colección gestiona la lógica de los desafíos (quizzes) que el usuario debe superar para obtener recompensas. Cada documento representa un reto específico asociado a un monumento.

Campo	Tipo de Dato	Descripción
titulo	String	Nombre del desafío (ej: "Desafío del Anfiteatro").
puntos_interes_id	String (Ref)	ID del documento de la colección puntos_interes al que pertenece la misión.
puntos_premio	Number	Recompensa que se sumará al perfil del usuario tras superar el reto.
preguntas	Array (Mapas)	Lista de preguntas que componen el quiz.

Anatomía de cada pregunta (dentro del Array):

- **pregunta_X** (String): El enunciado del reto.
- **respuestas** (Array): Lista de 3 opciones de texto para que el usuario elija.
- **indice_correcto** (Number): Posición (0, 1 o 2) dentro del array de respuestas que marca la opción válida.

6.7. Colección: resultado

Esta colección almacena el registro histórico de las rutas finalizadas por los usuarios. Es esencial para la analítica del progreso y para evitar la duplicidad de recompensas.

Campo	Tipo de Dato	Descripción
id_usuario	String (Ref)	Referencia al documento del usuario que completó la actividad.
id_ruta	String (Ref)	Referencia a la ruta específica que se ha finalizado.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

Campo	Tipo de Dato	Descripción
fecha_ruta_completada	Timestamp	Registro exacto del momento en que el usuario terminó la misión.
puntos_interes_visitados	Number	Contador de cuántos monumentos dentro de la ruta fueron validados con éxito.
puntos_partida	Number	Puntuación que obtuvo el usuario específicamente en esta sesión de juego.
tiempo_empleado	String	Cronometraje del tiempo que le tomó al turista completar todo el recorrido.

6.8. Modelo Entidad-Relación (ERD)

Aunque **Cloud Firestore** es una base de datos NoSQL y no requiere un esquema rígido de claves foráneas, nuestro diseño mantiene una integridad lógica relacional.

- **Relaciones principales:**
 - **Usuarios ↔ Resultados:** Relación de 1 a muchos (1:N). Un usuario puede tener múltiples registros de rutas completadas.
 - **Ciudades ↔ Rutas:** Relación de 1 a muchos (1:N). Una ciudad agrupa diferentes itinerarios.
 - **Rutas ↔ Puntos de Interés:** Relación de muchos a muchos (N:M). Un monumento puede formar parte de varias rutas temáticas.
 - **Puntos de Interés ↔ Misiones:** Relación de 1 a 1 (1:1). Cada monumento tiene un desafío específico asociado.

6.9. Migraciones y Gestión de Esquema

Al ser una base de datos **Serverless**, no utilizamos archivos de migración SQL tradicionales. El esquema se gestiona mediante:

1. **Reglas de Seguridad (firestore.rules):** Definen quién puede leer y escribir en cada colección, actuando como una "capa de validación" del esquema.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

2. **Modelos de Datos en Dart:** En la capa Data de la app, los archivos model.dart se encargan de transformar los documentos JSON de Firebase en objetos del lenguaje, asegurando que los tipos (String, Int, Geopoint) sean correctos.

6.10. Seed y Datos de Prueba

Para garantizar que la aplicación sea funcional desde el primer despliegue, hemos implementado una estrategia de Seeding:

- **Carga inicial:** Hemos poblado la base de datos con contenido real de la ciudad de **Mérida**, incluyendo monumentos icónicos como el Teatro y el Anfiteatro.
- **Usuarios de Test:** Contamos con perfiles de prueba con el campo isAdmin: true para testear el acceso a funciones restringidas.
- **Imágenes:** Todos los campos image_url apuntan a recursos optimizados alojados en **Firebase Storage** o repositorios de confianza (GitHub Assets), asegurando que la carga visual de la app sea fluida durante la evaluación.
- **Validación de QR:** Los códigos almacenados en la colección puntos_interes (ej: MER_ANFITEATRO_001) tienen sus correspondientes códigos QR físicos generados para las pruebas de campo.

7. Pruebas del sistema

7.1 Objetivos de las pruebas

El objetivo de las pruebas realizadas durante esta fase del proyecto ha sido verificar que el flujo principal de la aplicación funciona correctamente y que la comunicación con los servicios externos (Firebase Authentication y Cloud Firestore) se realiza sin errores.

Las pruebas se han centrado principalmente en validar el funcionamiento del **Producto Mínimo Viable (PMV)**, comprobando que el usuario puede registrarse, iniciar sesión, seleccionar una ruta y completar misiones dentro del sistema gamificado.

Estas pruebas permiten confirmar que las funcionalidades implementadas hasta esta evaluación son operativas y que el sistema puede continuar evolucionando hacia la versión final.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026	Tutor/a: María Mercedes Martínez Frago	

7.2 Entorno de pruebas

Las pruebas del sistema se han realizado utilizando distintos entornos de ejecución para asegurar el correcto funcionamiento de la aplicación.

Dispositivos utilizados:

- Dispositivo móvil Android físico utilizado como dispositivo de prueba conectado al entorno de desarrollo.
- Emulador de Android Studio configurado para simular diferentes dispositivos Android.

El uso de ambos entornos permitió verificar el comportamiento de la aplicación tanto en dispositivos reales como en entornos virtuales de desarrollo.

7.3 Tipos de pruebas realizadas

Durante el desarrollo del MVP se han realizado principalmente pruebas funcionales, centradas en validar el comportamiento del sistema desde el punto de vista del usuario.

Las funcionalidades comprobadas han sido las siguientes:

Autenticación de usuario

Se ha verificado el correcto funcionamiento del sistema de autenticación mediante Firebase Authentication.

Pruebas realizadas:

- Registro de nuevos usuarios.
- Inicio de sesión con credenciales válidas.
- Comprobación de error al introducir credenciales incorrectas.
- Creación automática del documento de usuario en Firestore tras el registro.

Resultado esperado:

El usuario puede registrarse y acceder correctamente a la aplicación.

Resultado obtenido:

Funcionamiento correcto.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

Selección de ciudad

Se ha comprobado la correcta carga de ciudades disponibles desde la base de datos.

Pruebas realizadas:

- Visualización del listado de ciudades disponibles.
- Selección de una ciudad por parte del usuario.
- Carga dinámica de las rutas asociadas a dicha ciudad.

Resultado esperado:

La aplicación debe mostrar únicamente las rutas pertenecientes a la ciudad seleccionada.

Resultado obtenido:

Funcionamiento correcto.

Selección de rutas

Una vez seleccionada la ciudad, se ha verificado la correcta carga de las rutas disponibles.

Pruebas realizadas:

- Visualización de las rutas asociadas a una ciudad.
- Selección de una ruta concreta.
- Inicio del flujo de misiones de la ruta.

Resultado esperado:

El usuario debe poder iniciar una ruta seleccionada.

Resultado obtenido:

Funcionamiento correcto.

Sistema de validación mediante QR

El sistema utiliza códigos QR como mecanismo de validación de presencia en cada punto de interés.

Pruebas realizadas:

- Escaneo de códigos QR generados por el equipo de desarrollo.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

- Validación del QR correspondiente a la misión activa.
- Desbloqueo de la misión tras la lectura correcta del QR.

Durante esta fase del proyecto se han utilizado **códigos QR de prueba generados internamente**, no ubicados físicamente en monumentos reales.

Resultado esperado:

El sistema debe validar el QR correcto y permitir continuar con la misión.

Resultado obtenido:

Funcionamiento correcto.

Sistema de misiones (Quiz)

Se ha probado el funcionamiento del sistema de preguntas asociado a cada misión.

Pruebas realizadas:

- Visualización de preguntas del quiz.
- Registro de respuestas del usuario.
- Avance entre preguntas de la misión.

Resultado esperado:

El usuario debe poder completar la misión respondiendo a las preguntas.

Resultado obtenido:

Funcionamiento correcto.

Flujo completo de una ruta

Se ha probado el flujo completo de una ruta dentro de la aplicación.

Flujo probado:

1. Registro o inicio de sesión del usuario.
2. Selección de ciudad.
3. Selección de ruta.
4. Escaneo del QR inicial.
5. Realización de la primera misión (quiz).

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

6. Escaneo del siguiente QR.
7. Repetición del proceso hasta completar todas las misiones.
8. Finalización de la ruta y retorno a la pantalla principal.

Resultado esperado:

El usuario debe poder completar todas las misiones de la ruta.

Resultado obtenido:

Funcionamiento correcto en el MVP actual.

7.4 Evidencias de las pruebas

Durante el desarrollo se han realizado pruebas funcionales directas mediante el uso del emulador de Android Studio y dispositivos físicos.

Las evidencias principales de estas pruebas se basan en la ejecución correcta de los flujos descritos anteriormente.

Para la defensa final del proyecto se contempla incluir:

- Capturas de pantalla de las pantallas principales del sistema.
- Demostración funcional en tiempo real de la aplicación.
- Vídeo corto del flujo principal de uso en caso de fallo durante la demostración en directo.

8. Problemas conocidos y limitaciones actuales

Durante el desarrollo del Producto Mínimo Viable (PMV) se han identificado una serie de limitaciones técnicas y aspectos que aún se encuentran en fase de mejora. Estas limitaciones no impiden el funcionamiento del sistema, pero representan áreas que deben optimizarse en futuras iteraciones del proyecto.

8.1 Integración del mapa

Uno de los principales aspectos pendientes en la aplicación es la integración del sistema de navegación mediante mapa.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026	Tutor/a: María Mercedes Martínez Fragoso	

Inicialmente se intentó implementar un sistema de navegación que se nos proporcionó. Sin embargo, durante el proceso de integración surgieron diversos problemas técnicos relacionados con su implementación dentro de la arquitectura actual del proyecto.

Debido a estas dificultades y al tiempo disponible para esta fase del desarrollo, decidimos no utilizar este sistema en la versión actual para evitar inestabilidad en la aplicación.

Actualmente el flujo de las misiones se realiza directamente mediante **escaneo de códigos QR**, lo que permite validar la “presencia” del usuario en los puntos de interés sin depender del sistema de navegación.

Para la versión final del proyecto se tiene previsto implementar un sistema de navegación basado en **Google Maps API**, que permitirá mostrar la ubicación del usuario, los puntos de interés y facilitar la orientación durante las rutas.

8.2 Ajustes de cálculos lógicos del sistema

Durante las pruebas realizadas se han detectado algunos comportamientos que requieren ajustes en la lógica interna del sistema.

Estos aspectos afectan principalmente a:

- cálculo de puntuaciones tras completar misiones
- control del progreso de las rutas
- gestión del flujo completo al finalizar una ruta

Aunque estas funcionalidades funcionan en el MVP actual, se están revisando para mejorar la coherencia de los resultados mostrados al usuario y asegurar una correcta actualización del progreso dentro del sistema gamificado.

8.3. Persistencia y sincronización de datos

Algunas operaciones relacionadas con la persistencia de datos en Cloud Firestore todavía se encuentran en fase de mejora.

Entre los aspectos que se están revisando se incluyen:

- actualización correcta de los puntos acumulados del usuario
- registro de rutas completadas

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

- sincronización del perfil del usuario tras finalizar misiones

Estas operaciones funcionan de forma básica en la versión actual, pero requieren optimización para garantizar mayor robustez en futuras versiones.

8.4 Validación de QR en entorno real

Durante esta fase del proyecto, las pruebas del sistema de validación se han realizado utilizando códigos QR generados internamente por el equipo de desarrollo.

Esto implica que todavía no se ha realizado una validación completa en un entorno turístico real con códigos QR ubicados físicamente en monumentos.

Las pruebas en campo permitirán validar aspectos como:

- condiciones de iluminación
- velocidad de escaneo
- comportamiento en exteriores
- precisión de la secuencia de validación de misiones

8.5 Sistema de progresión y rangos

El sistema de progresión basado en puntos y rangos está definido en el modelo de datos del proyecto, pero aún no se encuentra completamente implementado en la interfaz de usuario.

Actualmente el sistema permite completar misiones, pero todavía se están desarrollando las funcionalidades relacionadas con:

- actualización visual del rango del usuario
- visualización completa del sistema de logros
- feedback visual tras completar rutas

Estas mejoras se incorporarán en próximas fases del desarrollo.

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

8.6 Limitaciones del uso de Firebase

El proyecto utiliza **Firebase como backend principal**, lo que permite gestionar autenticación de usuarios y almacenamiento de datos de forma sencilla.

Sin embargo, durante el desarrollo se ha identificado que algunas funcionalidades avanzadas de Firebase requieren planes de pago o configuraciones adicionales que exceden el alcance del proyecto en esta fase.

Debido a estas limitaciones, ciertas funcionalidades potenciales no han podido implementarse completamente en el MVP, como por ejemplo:

- algunas herramientas avanzadas de analítica
- servicios adicionales de backend
- funcionalidades de escalado o automatización

Estas características podrían incorporarse en versiones futuras del proyecto si se dispone de un entorno de producción o de un plan de servicios más avanzado.

9. Próximos pasos para la defensa final

El objetivo principal para la defensa final del proyecto es completar el desarrollo del sistema y consolidar todas las funcionalidades del Producto Mínimo Viable (PMV), asegurando que la aplicación funcione de forma estable y permita demostrar el flujo completo de uso durante la presentación.

Para ello hemos definido una serie de mejoras y tareas pendientes que se implementarán en las siguientes fases del desarrollo.

9.1 Integración del sistema de navegación con mapas

Uno de los pasos más importantes será la implementación de un sistema de navegación mediante mapas que permita guiar al usuario entre los diferentes puntos de interés de una ruta.

Para la fase final del proyecto se prevé integrar **Google Maps API**, lo que permitirá:

- mostrar la ubicación del usuario en tiempo real
- visualizar los monumentos o puntos de interés en el mapa
- mejorar la orientación del usuario durante las rutas

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

Esta funcionalidad complementará el sistema de validación mediante códigos QR ya existente.

9.2 Optimización del sistema de puntuación y progreso

Otra de las mejoras previstas consiste en revisar y optimizar la lógica del sistema de puntuación y progreso del usuario.

Los objetivos principales serán:

- asegurar el cálculo correcto de los puntos obtenidos en cada misión
- actualizar correctamente el progreso del usuario en Firestore
- mejorar el control del flujo al finalizar una ruta completa

Esto permitirá que el sistema de gamificación funcione de forma más consistente y fiable.

9.3 Implementación completa del sistema de rangos

El proyecto contempla un sistema de progresión basado en rangos que permite visualizar el progreso del usuario dentro de la aplicación.

Para la defensa final se prevé completar la implementación de este sistema, incluyendo:

- cálculo automático del rango según los puntos acumulados
- visualización del rango dentro del perfil del usuario
- feedback visual tras completar rutas o misiones

9.4 Mejoras en la persistencia de datos

También se prevé mejorar la gestión de los datos almacenados en **Cloud Firestore** para garantizar una mayor estabilidad del sistema.

Entre las mejoras previstas se incluyen:

- registro completo de rutas finalizadas
- actualización correcta de puntos acumulados

	Andres Fernandez Exposito Diego Vivas Paredes Joel Manuel García	13/03	V 1
	0492 · Proyecto DAM · 2025/2026 Tutor/a: María Mercedes Martínez Fragoso		

- mejora de la sincronización de datos del perfil de usuario

9.5 Mejoras en la experiencia de usuario

De cara a la defensa final, también se realizarán mejoras en la interfaz de usuario para ofrecer una experiencia más clara e intuitiva.

Entre estas mejoras tendríamos:

- optimización de la navegación entre pantallas
- mejoras visuales en el flujo de misiones
- incorporación de elementos visuales relacionados con la gamificación (puntos, progreso, rangos)

9.6 Preparación de la demostración final

Finalmente, se preparará la aplicación para garantizar que la demostración durante la defensa del proyecto pueda realizarse de forma fluida.

Esto incluirá:

- preparación de datos de prueba en la base de datos
- revisión completa del flujo principal de uso de la aplicación
- pruebas finales en dispositivos reales

Además, se prepararán capturas de pantalla o vídeos cortos que permitan mostrar el funcionamiento de la aplicación en caso de que surja algún problema técnico durante la demostración.